

Report for the Course on Cyber-Physical Systems and IoT Security

Authentication of IoT Device and IoT Server Using Secure Vaults

Gianluca Bresolin

Objectives

In this report, the results of a Rust and Elixir implementation of the *Secure Vault Authentication Protocol* for IoT device–server communication are presented.

As described in the original paper, the *Secure Vault* scheme relies on a shared set of n keys of m bits each (the "secure vault"), pre-provisioned on both the IoT device and the server at deployment time, to mutually authenticate the two parties without relying on a single static shared secret. Authentication is performed through a three-way challenge–response exchange ($M1$ – $M4$), in which each party proves knowledge of a temporary key derived by XOR-ing a subset of vault entries indicated by a challenge, rather than exchanging any vault key directly. A successful authentication session also establishes a shared session key, derived from random values contributed by both device and server, used to protect all subsequent communication for the duration of that session. At the end of each session, the secure vault itself is updated on both sides via an HMAC computed over the exchanged session data, so that the set of keys used for future authentications evolves over time without requiring any additional message exchange. This continuous rotation of the vault prevents the kind of exposure that static, unchanging keys would create over repeated sessions, and, as shown in the original paper under the random oracle model, ensures that even if an attacker recovers part of the current vault through a side-channel attack, this does not allow prediction of the next vault's values, thereby limiting the impact of a partial compromise to a single session. This is the mechanism implemented and evaluated in this report.

The objective of this report is to demonstrate how our Rust (on *IoT Device* side) and Elixir (on server side) implementation of the *Secure Vault Authentication Protocol* allows an *IoT Device* and an *IoT Server* to mutually authenticate and to repeat this authentication process over time, with the secure vault being consistently and independently updated on both sides after each session.

Regarding the threat model, we assume that the initial provisioning phase of the *IoT Device* and the *Server*, during which the initial secure vault is shared between the two parties, has not been compromised, and that the initial vault keys are therefore unknown to any attacker.

To demonstrate the effectiveness of this implementation, a simple scenario involving two emulated entities is set up: one *IoT Device* and one *Server*, communicating over multiple successive authentication sessions. This setup allows observation of the full challenge–response exchange at each session, the derivation and usage of the session key for that session's communication, and the evolution of the secure vault contents across sessions.

The remainder of this report is organized as follows: the '*System Setup*' section describes the simulation tools used and implemented for the experiments; the '*Experiments*' section presents the methodology adopted to carry out our experiments. Finally, the '*Results and Discussions*' section outlines the results and our considerations regarding the experiments and the implemented authentication mechanism.

System Setup

This section describes the setup adopted to implement and test the proposed authentication scheme between the *Server* and the *IoT Device*. Both the *Server* and the *IoT Device* communicate through *CoAP* (*Constrained Application Protocol*), a lightweight UDP-based protocol designed for constrained devices and networks, which mirrors the kind of transport realistically available to IoT devices in the field. The section is organized into two parts: the first describes the environment and technologies used to implement the *Server*, while the second focuses on the emulation of the *IoT Device*, detailing the constraints and design choices adopted to reproduce a realistic embedded scenario.

1. Server

The server-side infrastructure is entirely orchestrated through *Docker Compose*, which coordinates the different components needed to run the authentication server together with the secure key storage it relies on. As assumed by the reference paper, cryptographic keys are not kept in the server's own memory or filesystem, but are stored inside a secure vault. To satisfy this assumption in a realistic way, we adopted *HashiCorp Vault* as the secure key storage backend.

Communication with `vault` is secured via *TLS*. Before `vault` starts, a dedicated `tls-init` service generates a self-signed certificate and private key (`vault-cert.pem`, `vault-key.pem`), valid for the vault hostname as well as localhost and `127.0.0.1`, ensuring that all subsequent traffic to the `vault` API is encrypted.

The `vault` service itself runs the official *HashiCorp Vault* image, configured through `vault-config.hcl`. `vault` is configured with an in-memory storage backend, with a TCP listener bounded to port `8200` and configured with the certificate and key generated by `tls-init` service, exposing the `vault` API exclusively over HTTPS. The `IPC_LOCK` capability is granted to the container to allow `vault` to lock memory pages and prevent secrets from being swapped to disk.

Since `vault` starts in a sealed state and cannot serve or store secrets until initialized and unsealed, a separate `vault-init` service is responsible for this bootstrap phase. This service waits for `vault` to become reachable, then performs a single-key initialization, a simplification suitable for a controlled test environment.

The resulting unseal *key* and *root token* are extracted from the initialization output and used to unseal `vault` programmatically. Once unsealed, the dedicated policy `iot-device-policy` is created, granting only create, read and update capabilities on the `secret/data/iot_device/*` path. A service token scoped to this policy is then generated and shared with the server, so that it can later authenticate to Vault with least-privilege access, without ever using the root token.

Finally, the `server` service builds the actual authentication IoT server from a custom

Dockerfile, and only starts once `vault-init` has completed successfully, ensuring that `vault` is already initialized, unsealed and provisioned with the correct policy and token. The `server` connects to `vault` over *HTTPS* using the shared *CA certificate* and retrieves the previously generated service token. Runtime parameters, including the set of registered *IoT Device* identifiers (`IOT_UIDS`) and pre-shared keys (`SERVER_KEYS`), are injected through environment variables loaded from a `.env` file. All services are attached to a dedicated *Docker network* (`iot_net`) with static IP addressing, with the server exposing port 5683 for *UDP* connections, the standard port for the *CoAP* protocol.

The server itself is implemented in *Elixir* and structured as a standard Mix/OTP application, which handles massive concurrency through lightweight, isolated processes, a property well suited to a server.

On startup, the Application module reads the environment configuration previously mentioned, including a numeric parameter `P` used by the authentication scheme, and finally starts a Supervisor¹ with a one-for-one restart strategy, supervising a GenServer² (`Server`) that holds the authentication logic and runtime state. During its `init/1` callback, this GenServer stores the configured keys into the secure vault on behalf of each registered device identifier through a dedicated `SecureVault` interface, and initializes its internal state, which tracks active authentication sessions, established sessions, the number of available keys, and the protocol parameter `P`, which stands for the number of indexes used by the authentication scheme for the key session generation.

Communication with the *IoT Device* takes place over *CoAP*, using the *Erlang* `gen_coap` library (notice that *Elixir* and *Erlang* are interoperable within the same execution environment, allowing seamless integration). On activation of the `Server`, the application starts the `gen_coap` server, which registers three resource handlers listening for *UDP CoAP* requests: `auth/m1`, `auth/m3` and `data`. Incoming *POST* requests are dispatched by a `Server.CoapHandler` module, implementing the `coap_resource behaviour`³, which routes each request to the corresponding server-side processing function (`process_m1`, `process_m3`, `process_data`) based on the resource path, returning the resulting payload as the *CoAP* response. All other *CoAP* methods are explicitly disabled, since the protocol only requires posting data to these three endpoints.

Cryptographic primitives used by the server are implemented in a dedicated `Server.Crypto` module, which wraps *Erlang's* built-in `:crypto` library. This module exposes *HMAC* computation (*HMAC-SHA256* by default) and *AES-256-CBC* encryption/decryption with *PKCS7* padding, used respectively for message

¹ An OTP process responsible for starting, monitoring, and automatically restarting its child processes according to a defined strategy, ensuring fault tolerance.

² A generic OTP behaviour for implementing stateful server processes, handling synchronous and asynchronous requests through defined callback functions.

³ An *Erlang/Elixir* construct defining a standard set of callback functions that a module must implement, enforcing a common interface across different implementations.

authentication and for protecting the confidentiality of exchanged payloads.

2. IoT Device

The *IoT Device* is emulated as a bare-metal application, written in *Rust* with `#[no_std]` and `#![no_main]`, targeting `aarch64-unknown-none`. This choice reflects the resource constrained nature of real *IoT* hardware: no operating system, no standard library, and full control over memory layout, boot sequence and hardware initialization, which are otherwise hidden behind OS abstractions on general-purpose systems.

The device is emulated with *QEMU*, using the generic `virt` machine model with a *max* CPU and 128 MiB of RAM. Since there is no OS to load the binary, *QEMU* boots the compiled ELF directly as a kernel image via `-kernel`. Two *virtio* devices are attached to the emulated machine: `virtio-net-device`, providing network connectivity, and `virtio-rng-device`, providing a source of hardware entropy, both needed to satisfy requirements of the authentication scheme.

At boot, execution starts from a small assembly routine (`boot.S`) that installs the *exception vector table*, zeroes the `.bss` section, sets up the stack pointer and jumps into the Rust entry point (`_start_main`). The MMU (*Memory Management Unit*) is configured manually and a minimal exception handler is installed, which, since no recovery is implemented for faults in this constrained environment, simply halts the core on a synchronous exception. Cross-compilation of the assembly boot code and linking against a custom linker script (`link.ld`, which defines the memory layout, including a dedicated DMA region required by the *virtio* drivers) are handled through a `build.rs` build script, invoked automatically by *Cargo*.

Hardware access to the *network* and *RNG virtio* devices is achieved through the `virtio-drivers` crate, which is hardware-agnostic and requires the platform to provide a `HAL` implementation, which in this project is provided via `MyHal` structure, managing *DMA* buffer allocation out of the reserved *DMA* region and by mapping shared buffers directly. Both *virtio* devices are located by scanning the fixed *MMIO* address space exposed by the `virt` machine, searching for the slot exposing the expected device type. The *network* device is then wrapped so that it implements the `Device` trait expected by `smoltcp`, the network stack used for the *IP/UDP* layer, while the *RNG* device is wrapped to implement the `Rng` trait, so that it can be used as an entropy source wherever randomness is required.

Since implementing a full *CoAP* library was unnecessary for the scope of this project, *CoAP* message encoding and decoding are implemented manually, exposing only the subset of the protocol needed to communicate with the server, i.e. building non-confirmable *POST* requests with *URI-path* options and a payload, and parsing responses to extract the payload from received *CoAP* packets. This logic, together with a `CommunicationManager` module, built on top of a `smoltcp` *UDP* socket, handles sending requests to the server, retransmission on timeout, and non-blocking

reception of responses.

Regarding cryptographic operations on the *IoT Device*, established *Rust* crates (`aes`, `cbc`, `hmac`, `sha2`) are used, providing *AES-256-CBC* encryption/decryption and *HMAC-SHA256*, mirroring the primitives used on the server side.

As assumed by the reference paper, cryptographic keys are kept in an encrypted form on the *IoT Device* memory. This is managed by the `DeviceKeyStore` struct, which keeps the pre-shared keys encrypted in memory using a separate, device-specific storage key, decrypting them only when needed.

The initial set of keys, the storage key, the device identifier and the protocol parameter `P` are injected at build time as environment variables, parsed at runtime through a minimal custom *JSON/base64* parser, allowing potentially each emulated device instance to be provisioned independently.

As with the *Server*, the whole *IoT Device* environment is containerized. A dedicated `Dockerfile` installs the `aarch64-unknown-none` *Rust* target and the cross-toolchain needed to build the boot assembly, compiles the firmware at image build time, and starts *QEMU* through an `entrypoint.sh` script. Since `virtio-net` requires a network interface to attach to, this script creates a *TAP* interface (`tap0`) inside the container and configures *NAT*/forwarding rules via *iptables*, bridging the emulated device's virtual network to the container's own network interface.

Finally, the `iot_device` service is attached to the same *Docker network* as the server (`iot_net`), with a static IP address, allowing the emulated device to reach the server's *CoAP* endpoint over *UDP* exactly as it would over a real network.

Experiment

To validate the proposed authentication scheme, the experiment is organized as a two-node scenario involving a single *IoT Device* (`iot_device`) and the *IoT Server* (`server`), with the whole system orchestrated through *Docker Compose* as described in the previous section.

The whole authentication scenario is started through a `run_experiment.sh` script, which is responsible for generating the initial set of pre-shared keys before the two containers are even built. The script prompts for the number of keys `n` to generate, then produces `n` random 32B keys (base64-encoded via `openssl rand -base64 32`), together with a separate 32B storage key (`SKEY`) used by the *IoT Device* to encrypt its local key store, as described in the previous section. These values, along with the resulting key count, are written to a `.env` file, which is then read both by the shell environment and by *Docker Compose*, and finally triggers `docker compose up --build` to bring up the whole scenario.

Through this `.env` file, the same set of keys (`KEYS / SERVER_KEYS`) is consistently injected as build arguments into both the `server` and `iot_device` containers, ensuring that the two parties start the experiment with a shared, identical key pool, as required by the authentication scheme. The server container additionally receives `IOT_UIDS`, fixed to "1", identifying the single *IoT Device* participating in the experiment, and the protocol parameter `P` fixed to "3". Symmetrically, the `iot_device` container receives its own `IOT_UID`

(matching the identifier expected by the server), the `SKEY` used to protect its local key store, and the same parameter `P` again, fixed to "3". The two containers are connected on the `iot_net Docker network` with static IP addresses (172.20.0.2 for the server, 172.20.0.3 for the IoT Device).

Once the *IoT Device* and the *Server* have completed the authentication handshake and established a shared `session_key`, the *IoT Device* begins sending telemetry data to the server every 5 seconds. For the purpose of this experiment, this telemetry data is a simple placeholder counter, incremented by one at each successful send, rather than any real sensor reading. On the *Server* side, each authenticated session is given a fixed duration, set to 20 seconds in this scenario for the aim of observability, short enough to trigger session expiration and key renewal multiple times within a short observation window, allowing the full behavior described in the following subsection to be reproduced and inspected without having to wait for a realistically long session lifetime.

This section is divided into two parts: the first focuses on the *session key establishment phase*, validating that the *IoT Device* and the *Server* correctly authenticate each other and agree on a *shared session key*; the second focuses on *session expiration* and *changing the secure vault*, where both parties correctly detect the timeout and transition to a fresh authentication round using updated keys.

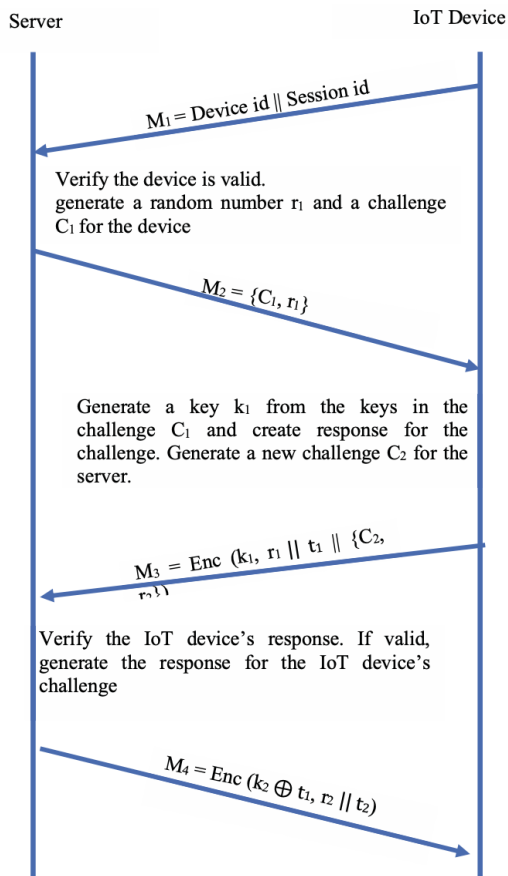
1. Session Key Establishment

On the *IoT Device* side, authentication is triggered by `auth_manager:init_auth_session()` in the `AuthManager` structure. The device generates a random `session_id`, (64 bits, chosen to provide sufficient entropy to avoid collisions in a real scenario), builds `M1` as the concatenation of its `iot_uid` and `session_id`, and sends it to the server on `auth/m1` via `CommunicationManager::send_and_receive()`, which handles retransmission and blocks until a response (`M2` in this case) is received or the request times out. On the *Server* side, `Server.CoapHandler.process_m1` extracts the raw `M1` binary from the *CoAP* payload and forwards it to the *Server* `GenServer`. There, the callback to `process :process_m1` parses `iot_uid` and `session_id` from `M1`, and rejects the request if `iot_uid` is not registered. Otherwise, it randomly selects a subset `C1` of `P` key indices and generates a nonce `R1`. Starting from the keys associated with the given `iot_uid`, it selects the keys at the indices in `C1` and derives from them a session key `K1`, storing `{K1, R1}` in the `auth_sessions` map under `session_id` as key. The pair `{C1, R1}` is then sent back to the device as `M2`.

Upon receiving `M2`, the *IoT Device* extracts via `auth_manager:process_m2_build_m3()` the `C1` and the nonce `R1`, then reconstructs `K1` by XOR-ing together the keys stored locally at the indices in `C1`. It generates a fresh nonce `T1`, selects a new set `C2` of `P` unique random key indices, and generates the nonce challenge `R2`, which is kept for later verification against the server's response. The plaintext `{R1, T1, C2, R2}` is then encrypted under `K1`

using *AES-256-CBC*, forming M_3 , which is sent back to the server alongside the current `session_id`.

Back on the *Server* side, `Server.CoapHandler.process_m3` extracts the raw M_3 binary from the *CoAP* payload and forwards it to the *Server GenServer*. Upon receiving M_3 , the *Server* retrieves the pending session state for the given `session_id` and decrypts M_3 using K_1 , extracting R_1 , an *IoT Device*-generated nonce T_1 and its set of key indices C_2 , and a fresh challenge nonce R_2 . If the decrypted R_1 does not match the one stored for this session, authentication fails and the pending session is discarded. Otherwise, the server derives a new key K_2 from C_2 indices, generates its own nonce T_2 , and encrypts $\{R_2, T_2\}$ under a key obtained by XOR-ing K_2 with an expansion of T_1 , producing M_4 . Finally, it derives the `session_key` by XOR-ing T_1 and T_2 together and stores it as an established session under `session_id`, together with its expiry time, discarding the previous and temporary authentication state.



Finally, the *IoT Device* receives back M_4 and reconstructs K_2 by XOR-ing together the keys stored locally at the indices in C_2 , deriving the decryption key by XOR-ing K_2 with T_1 . It then decrypts M_4 under this key using *AES-256-CBC*, obtaining R_2 and the server's nonce T_2 . If the decrypted R_2 does not match the one generated earlier by the *IoT Device*, the process is aborted. Otherwise, the `session_key` is derived by XOR-ing T_1 and T_2 together, completing the mutual authentication and establishing the shared session key.

2. Session Expiration

After the `session_key` has been established, the *IoT Device* periodically sends telemetry data to the *Server*, encrypted under that key. To bound the lifetime of a `session_key`, each session is associated with an expiry time, after which it is no longer accepted. This check is made by the server upon each telemetry data send and when a session expires, the *Server*

triggers a renewal of the underlying pre-shared keys associated to that specific `iot_uid`, discards the previous session associated to the provided `session_id` and replies back to the *IoT Device* with a session timeout error instead of acknowledging the data. This way, the device becomes aware of the expiration and updates its own keys accordingly, by initiating a new authentication round with the renewed key pool.

Key renewal happens via the following mechanism: the current set of pre-shared keys is treated as a single secure vault value, which is updated based on the data exchanged between the *Server* and the *IoT Device* during the expired session. Specifically, an *HMAC* is computed over this exchanged data, using the concatenation of the current keys as the *HMAC* key, producing an output h of k bits. The current vault value is then split into equal k -bit partitions, padding with zeros at the end if its size is not evenly divisible by k . Each partition is XORed with h , additionally shifted by its own partition index. The resulting values are finally concatenated into a single bitstring, which is then re-split into n blocks of m bits, regenerating the original number of keys at their original size forming the new set of keys that replaces the previous vault contents on both the *IoT Device* and the *Server*.

Results and Discussion

The project, including the full source code, is publicly available on *GitHub* at <https://github.com/GianlucaBresolin/MultiKeysAuthIoT>. Instructions to reproduce the experiment are provided in the repository under [INSTRUCTIONS.md](#).

In this project, we successfully implement the described authentication protocol that has led to the successful establishment of a mutual `session_key` between an *IoT Device* emulated in *QEMU* and an *Elixir IOT Server*. Thanks to this protocol, all this is achieved while maintaining several structural security strengths: because the encryption key used in each exchange is derived by XORing multiple keys from the secure vault, no single compromised key (or the derived session key itself) reveals the other keys still held in the vault, preventing an attacker from reconstructing the full vault or cloning the device even after a successful *side-channel* compromise. By being re-keyed after every session using an *HMAC* over the data exchanged during that session, each new set of keys is cryptographically independent of the previous one, closing off dictionary and replay attacks without requiring any extra message exchange between the *IoT Device* and *Server*. Finally, mutual authentication itself is enforced through a challenge-response exchange in both directions using distinct, randomly chosen index sets, which prevents a *man-in-the-middle* from impersonating either party, since the derived session key is never transmitted in the clear.

As proof, the following terminal snapshot presents the exchange of authentication messages (M1 through M4) between the *IoT Device* and the *Server*, leading to the successful establishment of the shared `session_key`.

```

iot_device-1 | IoT Device: starting up
iot_device-1 | Tx M1:
iot_device-1 | - IOT_UID: 01
iot_device-1 | - SESSION_ID: de362dfc252c9810
server-1 | Server initialized with SecureVault for 1 IoT devices.
server-1 | Rx M1:
server-1 | - IOT_UID: 1
server-1 | - SESSION_ID: 1195754242723100382
server-1 |
server-1 | Tx M2:
iot_device-1 | Rx M2:
server-1 | - C1: [6, 7, 13]
iot_device-1 | - C1: 06070d
server-1 | - R1: db54c3e312c23eaa
server-1 |
server-1 | - R1: db54c3e312c23eaa
iot_device-1 | - K1: c631be2236d67db791cb0d424fe6243f4252e84038a91379a63e2d45d0a23106
iot_device-1 | Tx M3:
iot_device-1 | - R1: db54c3e312c23eaa
iot_device-1 | - T1: c7028e29a1e62514
iot_device-1 | - C2: 001201
iot_device-1 | - R2: d91577885cd7ac31
server-1 | Rx M3:
server-1 | - SESSION_ID: 1195754242723100382
server-1 | - ENCRYPTED_M3: ab49df68bf522f345c54e69ca9e8565f2329e54cbafbe08b6d2e7b004c75c57c08a7c6a55e1573c9508c76751f3ce4d3
server-1 | - R1: db54c3e312c23eaa
server-1 | - T1: c7028e29a1e62514
server-1 | - C2: [0, 18, 1]
server-1 |
iot_device-1 | Rx M4:
server-1 | Tx M4:
server-1 | - K2: 1490d4ec141fd5f026a03b37891399e7a798f1db768aaaadadda6a70ad58ad1c
server-1 | - T1: c7028e29a1e62514
server-1 | - R2: d91577885cd7ac31
server-1 | - T2: baa0c9a7173817ba
server-1 | - ENCRYPTED_M4: b25e427f91f4cb329a24bacb4e4c1c04e8c8816787c691ad80e8c9f81ce23edb8958bbcd5ad3c5088c117a25db339192
server-1 | - R2: d91577885cd7ac31
server-1 | SESSION_KEY: 7da2478eb6de32ae7da2478eb6de32ae7da2478eb6de32ae7da2478eb6de32ae
server-1 |
iot_device-1 | - T2: baa0c9a7173817ba
iot_device-1 | SESSION_KEY: 7da2478eb6de32ae7da2478eb6de32ae7da2478eb6de32ae7da2478eb6de32ae

```

As a consequence, the exchange of telemetry data under the established `session_key` can be observed in the following terminal snapshot, with the *Server* correctly decrypting and acknowledging each value sent by the *IoT Device*.

```

iot_device-1 | IoT Device: starting main loop
server-1 | Encrypted Data: <<247, 169, 131, 152, 166, 248, 79, 159, 188, 152, 55, 29, 11, 0, 52, 48, 122, 215, 11, 64, 133, 34, 69, 244, 75, 73, 156, 157, 112, 82, 207, 38>>
server-1 | Telemetry Data: <<0, 0, 0, 0, 0, 0, 0>>
server-1 |
iot_device-1 | Ack received from server.
server-1 | Encrypted Data: <<251, 88, 108, 37, 46, 93, 162, 130, 92, 226, 113, 131, 238, 111, 71, 70, 146, 38, 51, 217, 197, 197, 195, 151, 144, 100, 47, 91, 151, 18, 194, 1>>
server-1 | Telemetry Data: <<0, 0, 0, 0, 0, 0, 1>>
iot_device-1 | Ack received from server.
server-1 | Encrypted Data: <<233, 280, 156, 230, 200, 114, 10, 218, 190, 172, 113, 208, 47, 77, 70, 221, 225, 212, 151, 210, 20, 137, 64, 157, 183, 158, 155, 8, 168, 84, 176, 206>>
server-1 | Telemetry Data: <<0, 0, 0, 0, 0, 0, 2>>
iot_device-1 | Ack received from server.
server-1 | Encrypted Data: <<199, 191, 244, 172, 223, 160, 63, 166, 229, 21, 66, 117, 122, 179, 100, 50, 51, 96, 192, 200, 190, 88, 186, 77, 5, 138, 121, 2, 140, 230, 105, 31>>
server-1 | Telemetry Data: <<0, 0, 0, 0, 0, 0, 3>>
iot_device-1 | Ack received from server.

```

Finally, this last terminal snapshot shows the expiration of the active session, together with the resulting renewal of the pre-shared keys and the establishment of a new session between the *IoT Device* and the *Server*.

